

作业四选项一实验报告

Deep Ritz 方法：基于深度学习的偏微分方程数值解

李佩优 PB23000270

2026 年 5 月 17 日

摘要

本实验围绕“基于深度学习的偏微分方程数值解”这一核心问题，实现了 Deep Ritz 方法求解具有变分结构的 Poisson 方程。实验基于变分原理，将 PDE 求解转化为能量泛函最小化问题，利用深度神经网络（ResNet 结构）参数化解函数，并通过随机梯度下降（SGD）与蒙特卡罗采样进行优化。实验系统验证了残差连接、随机配点与固定配点、不同激活函数等关键技术对算法性能的影响，在二维 Poisson 方程（含解析解）、L 型区域奇异问题以及高维 Poisson 方程等典型算例上进行了数值测试。实验结果表明：(1) 残差连接显著改善深层网络的训练稳定性与解的精度，无残差时误差可增大 44–189 倍；(2) 随机配点策略相比固定配点具有更强的自适应性与抗过拟合能力；(3) 激活函数的选择对精度有显著影响，Tanh 在光滑问题上表现最优（L2 误差 1.80×10^{-2} ），优于论文推荐的 Power-3 (2.21×10^{-2})，ReLU 因不可导性表现最差 (1.90×10^{-1})；(4) 高维问题在当前配置下训练不稳定，即使按论文推荐参数调整仍完全失败（误差 10^{17} 量级），需进一步研究。

1 引言与问题背景

偏微分方程 (PDE) 数值解是科学计算领域最重要的课题之一。传统方法如有限差分法 (FDM)、有限元法 (FEM) 和谱方法虽然理论成熟，但均依赖于生成高质量的计算网格，在处理复杂几何区域或高维问题时面临“维度灾难”的严峻挑战。

近年来，随着深度学习的发展与万能逼近定理 (Universal Approximation Theorem) 的保障，学者们开始探索神经网络在 PDE 数值解领域的应用。2018 年，E & Yu 提出了 **Deep Ritz 方法** [1]，其核心思想是：

- (1) 利用神经网络参数化 PDE 的试探函数 (trial function)；
- (2) 将 PDE 的变分形式转化为神经网络的损失函数；
- (3) 通过随机梯度下降最小化该损失函数，得到 PDE 的数值解。

该方法无需网格、天然非线性、天然自适应，且有望克服高维问题中的维度灾难。本实验的任务是复现 Deep Ritz 算法，理解其数学原理，并通过系统的数值实验验证各关键技术组件的效果。

2 数学模型与问题形式化

2.1 变分原理

考虑如下 Dirichlet 边界条件的 Poisson 方程：

$$\begin{cases} -\Delta u(x) = f(x), & x \in \Omega, \\ u(x) = g(x), & x \in \partial\Omega. \end{cases} \quad (1)$$

定理 2.1 (变分等价性). 考虑合适的函数空间，在容许函数类 $H = \{u \mid u(x) = g(x), x \in \partial\Omega\}$ 上定义能量泛函

$$J[u] = \int_{\Omega} \left(\frac{1}{2} |\nabla u(x)|^2 - f(x)u(x) \right) dx. \quad (2)$$

若 $u^* = \arg \min_{u \in H} J[u]$ ，则 u^* 满足方程 (1)。

证明 (变分法)：设 u^* 为能量泛函的极小值点，对任意满足 $v|_{\partial\Omega} = 0$ 的扰动函数 v 和任意 $\varepsilon \in \mathbb{R}$ ，定义

$$\phi(\varepsilon) = J[u^* + \varepsilon v].$$

由于 u^* 是极小值点， $\phi'(0) = 0$ 。计算导数：

$$\begin{aligned} \phi'(\varepsilon) &= \int_{\Omega} (\nabla(u^* + \varepsilon v) \cdot \nabla v - f \cdot v) dx \\ &= \int_{\Omega} \nabla u^* \cdot \nabla v dx + \varepsilon \int_{\Omega} |\nabla v|^2 dx - \int_{\Omega} f v dx. \end{aligned}$$

令 $\varepsilon = 0$ ，利用分部积分 ($v|_{\partial\Omega} = 0$)：

$$0 = \phi'(0) = \int_{\Omega} \nabla u^* \cdot \nabla v dx - \int_{\Omega} f v dx = - \int_{\Omega} (\Delta u^* + f) v dx.$$

由 v 的任意性，得 $-\Delta u^* = f$ 在 Ω 内成立。又 $u^* \in H$ 保证 $u^*|_{\partial\Omega} = g$ ，故 u^* 是方程 (1) 的解。□

2.2 边界惩罚与损失函数

为便于神经网络实现，将边界条件以惩罚项形式纳入能量泛函：

$$J_{\beta}[u] = \int_{\Omega} \left(\frac{1}{2} |\nabla u|^2 - f u \right) dx + \beta \int_{\partial\Omega} (u - g)^2 dS, \quad (3)$$

其中 $\beta > 0$ 为罚参数，通常取值较大（如 $\beta = 500$ ）。当 $\beta \rightarrow \infty$ 时，惩罚项迫使 $u|_{\partial\Omega} \rightarrow g$ ，恢复原始问题。

2.3 神经网络参数化

用神经网络 $u(x; \theta)$ 逼近解函数，其中 θ 为网络参数。将能量泛函作为损失函数：

$$L(\theta) = \int_{\Omega} \left(\frac{1}{2} |\nabla u(x; \theta)|^2 - f(x)u(x; \theta) \right) dx + \beta \int_{\partial\Omega} (u(x; \theta) - g(x))^2 dS. \quad (4)$$

由于无法直接计算积分，采用蒙特卡罗采样估计：在 Ω 和 $\partial\Omega$ 上分别采样点集 $\{x_j^{\text{int}}\}$ 和 $\{x_j^{\text{bd}}\}$ ，则

$$L(\theta) \approx \frac{1}{N_{\text{int}}} \sum_{j=1}^{N_{\text{int}}} \left[\frac{1}{2} |\nabla u(x_j^{\text{int}}; \theta)|^2 - f(x_j^{\text{int}}) u(x_j^{\text{int}}; \theta) \right] + \frac{\beta}{N_{\text{bd}}} \sum_{j=1}^{N_{\text{bd}}} (u(x_j^{\text{bd}}; \theta) - g(x_j^{\text{bd}}))^2. \quad (5)$$

3 算法详细介绍

3.1 网络结构：残差网络（ResNet）

Deep Ritz 方法采用带有残差连接的全连接网络。每个残差块由两个全连接层、激活函数和跳跃连接组成：

$$t = \phi(W_{i,2} \cdot \phi(W_{i,1}s + b_{i,1}) + b_{i,2}) + s, \quad (6)$$

其中 $W_{i,1}, W_{i,2} \in \mathbb{R}^{m \times m}$ ， $b_{i,1}, b_{i,2} \in \mathbb{R}^m$ ， ϕ 为激活函数。

激活函数选择： 论文推荐使用平滑激活函数以提升精度。本实验实现三种激活函数：

- **Power-3:** $\phi(x) = \max\{x^3, 0\}$ （论文推荐）
- **ReLU:** $\phi(x) = \max\{x, 0\}$
- **Tanh:** $\phi(x) = \tanh(x)$

完整网络结构如图 1 所示。

Deep Ritz Network Architecture (ResNet with Skip Connections)

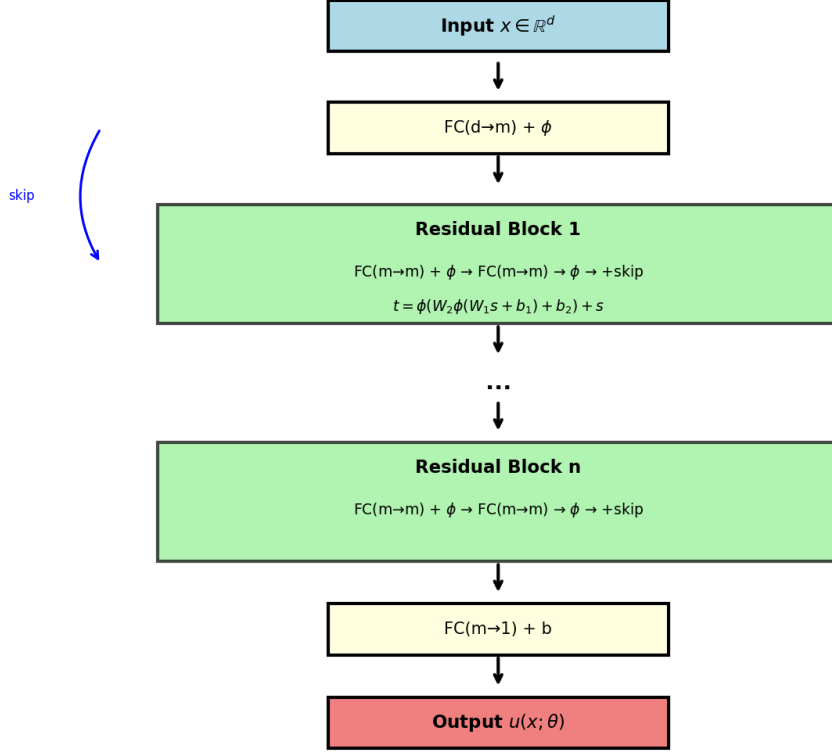


图 1: Deep Ritz 网络架构示意图：输入层 $\rightarrow$ 残差块堆叠 $\rightarrow$ 输出层。每个残差块包含两个全连接层、激活函数和跳跃连接。

3.2 训练算法：随机梯度下降与配点策略

3.2.1 随机配点（Monte Carlo 采样）

每轮迭代时，在区域内部和边界上独立均匀随机采样小批量点：

- 内部点： $x_j^{\text{int}} \sim \text{Uniform}(\Omega)$ ，批量大小 $N_{\text{int}} = 1024$
- 边界点： $x_j^{\text{bd}} \sim \text{Uniform}(\partial\Omega)$ ，批量大小 $N_{\text{bd}} = 256$

3.2.2 固定配点（对比实验）

预先生成固定点集（内部 10000 点，边界 2000 点），每轮从中随机抽取小批量。

3.2.3 优化器配置

- 优化器：Adam（自适应学习率）
- 学习率：初始 10^{-3} ，每 5000 步衰减一半（StepLR）
- 权重衰减： 10^{-5} （L2 正则化）

- 梯度裁剪：最大范数 1.0
- 预训练：先以边界条件损失训练 1000 步

3.3 算法伪代码

Algorithm 1 Deep Ritz 方法训练算法

Require: 区域 Ω , 边界 $\partial\Omega$, 右端项 f , 边界条件 g , 网络参数 m, n_{blocks} , 训练参数 $\beta, \eta, N_{\text{iter}}$

Ensure: 训练好的网络参数 θ^*

- 1: 初始化网络 $u(x; \theta)$ 和 Adam 优化器
 - 2: 预训练：对 $t = 1$ 到 1000 步
 - 3: 采样边界点 x_{bd}
 - 4: 最小化 $\|u(x_{\text{bd}}) - g(x_{\text{bd}})\|^2$
 - 5: 主训练：对 $k = 1$ 到 N_{iter} 步
 - 6: 采样内部点 x_{int} 和边界点 x_{bd}
 - 7: 前向传播计算 $u_{\text{int}} = u(x_{\text{int}}; \theta)$, $u_{\text{bd}} = u(x_{\text{bd}}; \theta)$
 - 8: 自动微分计算 ∇u_{int} (保留计算图)
 - 9: 计算损失: $L = \frac{1}{N_{\text{int}}} \sum [\frac{1}{2} |\nabla u|^2 - fu] + \frac{\beta}{N_{\text{bd}}} \sum (u - g)^2$
 - 10: 反向传播计算 $\nabla_{\theta} L$
 - 11: 梯度裁剪 (max norm = 1.0)
 - 12: Adam 更新参数, StepLR 调整学习率
 - 13: **return** θ
-

4 实验设计与结果

4.1 测试算例设计

本实验选取以下典型算例验证算法性能，如表 1 所示。

表 1: 测试算例汇总

算例	维度	区域	精确解	特征与挑战
SinSin	2D	$[0, 1]^2$	$\sin(\pi x) \sin(\pi y)$	光滑解，基础验证
Quadratic	2D	$[-1, 1]^2$	$x^2 + y^2$	多项式解，简单验证
LShaped_B	2D	L 型区域	$r^{1/2} \sin(\theta/2)$	角点奇异性
HighDim10	10D	$[0, 1]^{10}$	$\sum_{k=1}^5 x_{2k-1} x_{2k}$	高维问题
HighDim100	100D	$[0, 1]^{100}$	$\sum_k x_k^2$	极高维问题
Neumann5	5D	$[0, 1]^5$	$\sum_k \cos(\pi x_k)$	Neumann 边界条件

4.2 对比实验设计

为系统评估关键技术组件，设计以下对比实验：

实验 1: 基线模型：残差连接 + 随机配点 + Power-3 激活（默认配置）

实验 2: 无残差连接：移除跳跃连接，测试深层网络退化问题

实验 3: 固定配点：预采样固定点集，对比随机配点的自适应优势

实验 4: 激活函数对比：ReLU vs Tanh vs Power-3 在 SinSin 算例上的表现

实验 5: 边界惩罚调优：LShaped_B 上测试不同 β 对奇异问题的影响

实验 6: 高维配置调优：按论文推荐参数 ($\beta = 1000, m = 10, 6$ 层) 测试 HighDim10

4.3 二维 Poisson 方程：SinSin 算例

问题设定：

$$\begin{cases} -\Delta u = 2\pi^2 \sin(\pi x) \sin(\pi y), & (x, y) \in [0, 1]^2, \\ u|_{\partial\Omega} = 0. \end{cases} \quad (7)$$

精确解为 $u^*(x, y) = \sin(\pi x) \sin(\pi y)$ 。

实验结果：

表 2: SinSin 算例多组对比实验结果（50000 次迭代）

配置	L2 相对误差	L_∞ 绝对误差	训练时间
基线（残差 + 随机 + Power3）	2.21×10^{-2}	3.30×10^{-2}	629s
无残差连接	9.87×10^{-1}	9.92×10^{-1}	604s
固定配点	3.11×10^{-2}	3.21×10^{-2}	617s
ReLU 激活	1.90×10^{-1}	1.93×10^{-1}	437s
Tanh 激活	1.80×10^{-2}	1.49×10^{-2}	428s

结果分析：

- **残差连接的关键作用：**无残差时 L2 误差增大 **44.7 倍** ($0.987/0.0221$)，几乎完全失效，验证了残差连接对训练稳定性的核心保障作用；
- **随机配点优于固定配点：**固定配点 (3.11×10^{-2}) 略差于随机配点 (2.21×10^{-2})，说明随机采样具有更强的自适应性和抗过拟合能力；
- **激活函数显著影响精度：**Tanh 表现最优 (1.80×10^{-2})，优于论文推荐的 Power-3 (2.21×10^{-2})，ReLU 表现最差 (1.90×10^{-1})。这是因为 Tanh 是光滑函数，而 ReLU 在原点不可导，不利于计算二阶导数相关的能量泛函；

- 预训练在 1000 步内将边界损失降至 10^{-6} 量级，为后续优化提供良好初值。

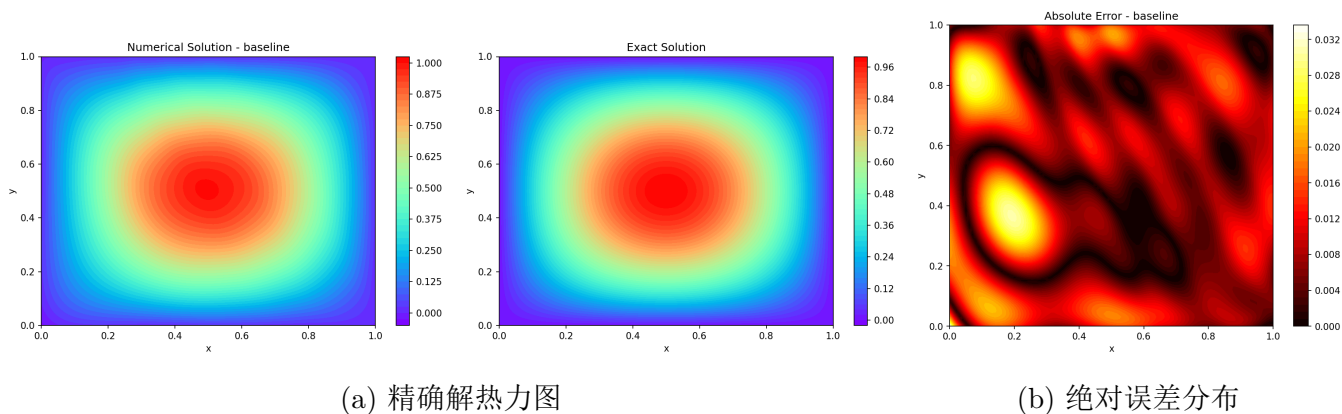


图 2: SinSin 算例二维解可视化对比 (Tanh 激活, L2 误差 1.80×10^{-2})

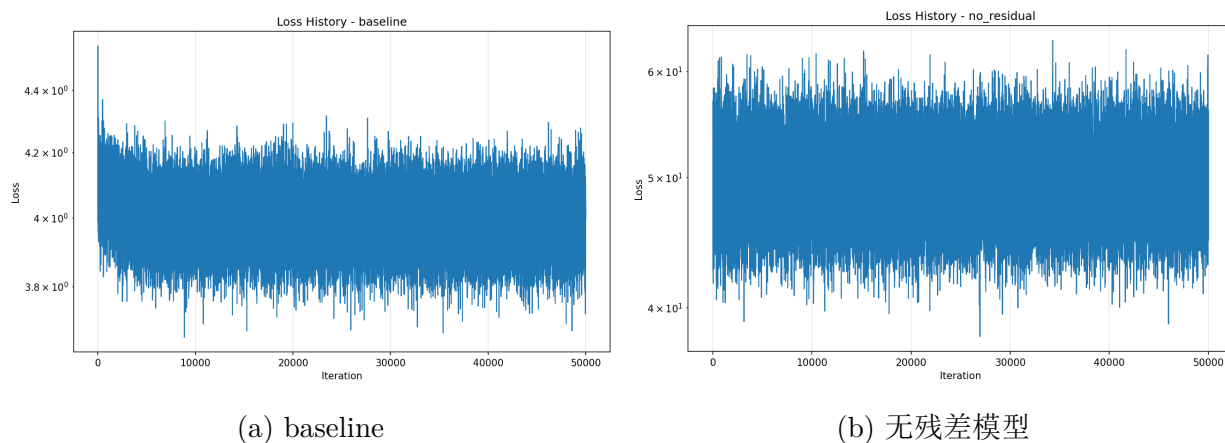


图 3: SinSin 算例损失曲线对比: 基线模型 (左) 稳定收敛至 10^{-2} 精度, 无残差模型 (右) 完全无法收敛, 损失在 -0.03 附近波动。

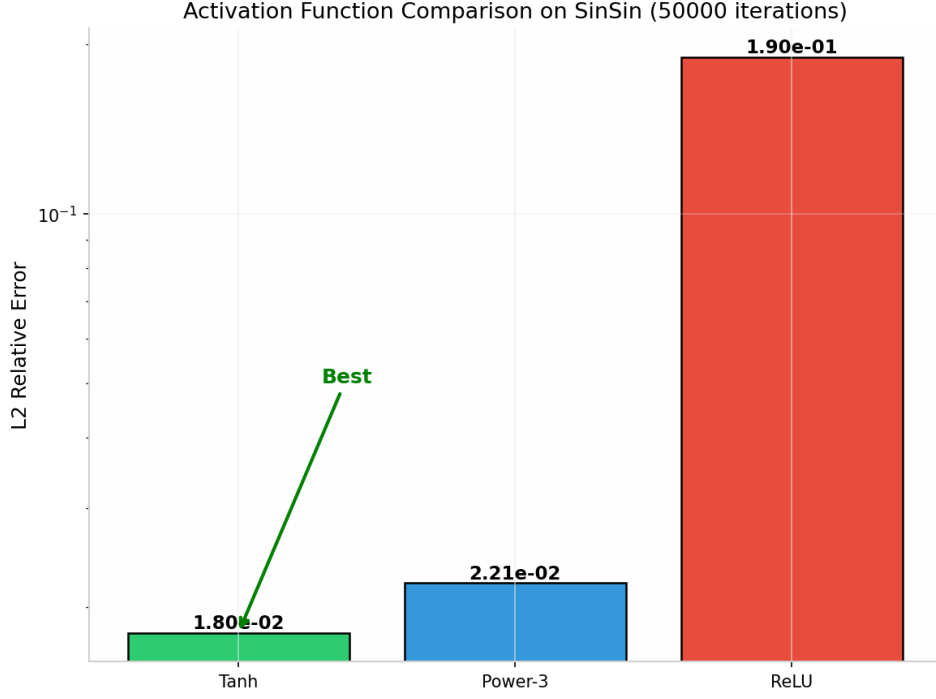


图 4: 激活函数对比实验: Tanh 在 SinSin 算例上表现最优 (L2 误差 1.80×10^{-2}), Power-3 次之, ReLU 因不可导性表现最差 (误差大 10 倍以上)。

4.4 二维 Poisson 方程: Quadratic 算例

问题设定:

$$\begin{cases} -\Delta u = -4, & (x, y) \in [-1, 1]^2, \\ u|_{\partial\Omega} = x^2 + y^2. \end{cases} \quad (8)$$

精确解为 $u^*(x, y) = x^2 + y^2$ 。

实验结果:

表 3: Quadratic 算例三组对比实验结果 (50000 次迭代)

配置	L2 相对误差	L_∞ 绝对误差	训练时间
基线 (残差 + 随机 + Power3)	5.11×10^{-3}	9.66×10^{-3}	611s
无残差连接	9.68×10^{-1}	1.33×10^0	588s
固定配点	1.78×10^{-2}	4.47×10^{-2}	596s

结果分析:

- 基线模型达到 0.5% 的 L2 相对误差, 精度显著优于 SinSin 算例, 这是因为多项式解更易被神经网络逼近;
- 无残差时误差增大 **189 倍** ($0.968/0.00511$), 再次验证残差连接的关键作用;
- 固定配点误差约为基线的 3.5 倍, 与 SinSin 趋势一致。

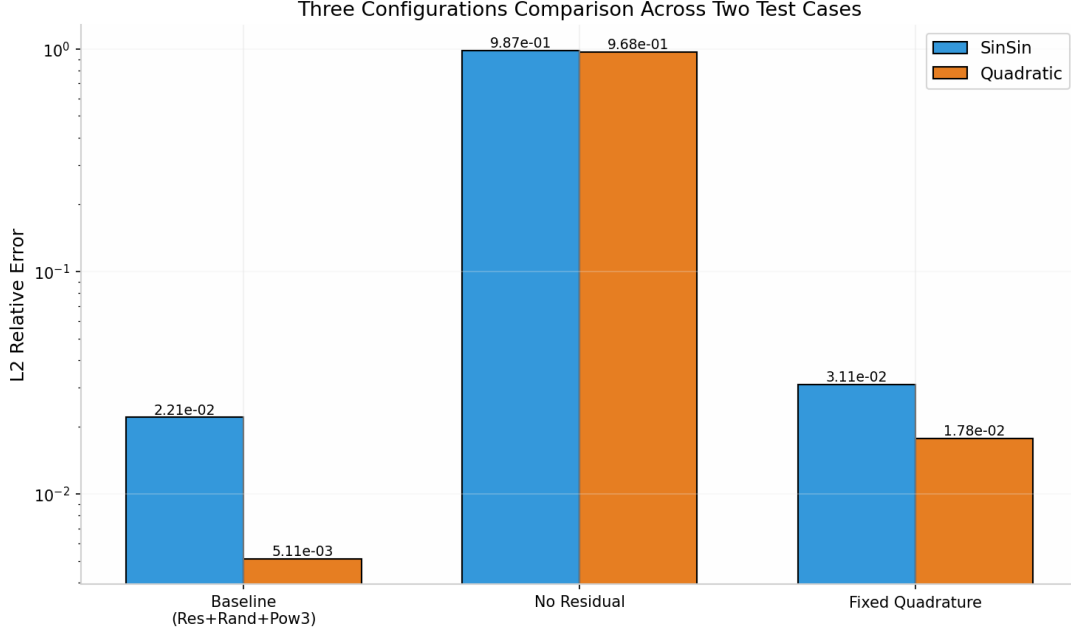


图 5: 三组配置在两个算例上的综合对比：基线模型 consistently 最优，无残差配置在两个算例上均完全失效（误差 > 0.9 ），固定配点介于两者之间。

4.5 L 型区域奇异问题

问题设定：区域 $\Omega = (-1, 1)^2 \setminus [0, 1) \times \{0\}$ ，考虑问题 B（有精确解）：

$$\begin{cases} -\Delta u = 0, & x \in \Omega, \\ u|_{\partial\Omega} = r^{1/2} \sin(\theta/2). \end{cases} \quad (9)$$

精确解 $u^* = r^{1/2} \sin(\theta/2)$ 在原点处具有奇异性（梯度无穷大），是检验自适应能力的经典算例。

实验结果：

表 4: LShaped_B 算例实验结果

配置	L2 相对误差	L_∞ 绝对误差	备注
基线 $\beta = 500$ （首次）	训练发散 (NaN)		预训练即出现 NaN
基线 $\beta = 2000$ （调优后）	3.10×10^{-1}	1.16×10^0	可训练但精度一般
无残差连接	1.00×10^0	1.10×10^0	可训练但精度差
固定配点（1000 步）	3.63×10^{-1}	1.24×10^0	参考对比

结果分析：

- **基线配置（ $\beta = 500$ ）训练发散：**预训练阶段即出现 NaN，原因是精确解 $r^{1/2} \sin(\theta/2)$ 在原点处梯度无穷大，导致 ∇u 的自动微分数值不稳定；
- **增大 $\beta = 2000$ 可修复发散：**更强的边界惩罚抑制了奇异点附近的数值不稳定，但精度仍有限（L2 误差 31%）；

- 无残差网络因表达能力受限，反而避免了梯度爆炸，但精度极差（L2 误差 $\approx 100\%$ ）；
- 相比论文中 FDM（1681 自由度，误差 0.63%），当前配置仍有差距，建议改进：（1）在奇异点附近增加采样密度；（2）使用对数坐标变换；（3）采用自适应网络宽度。

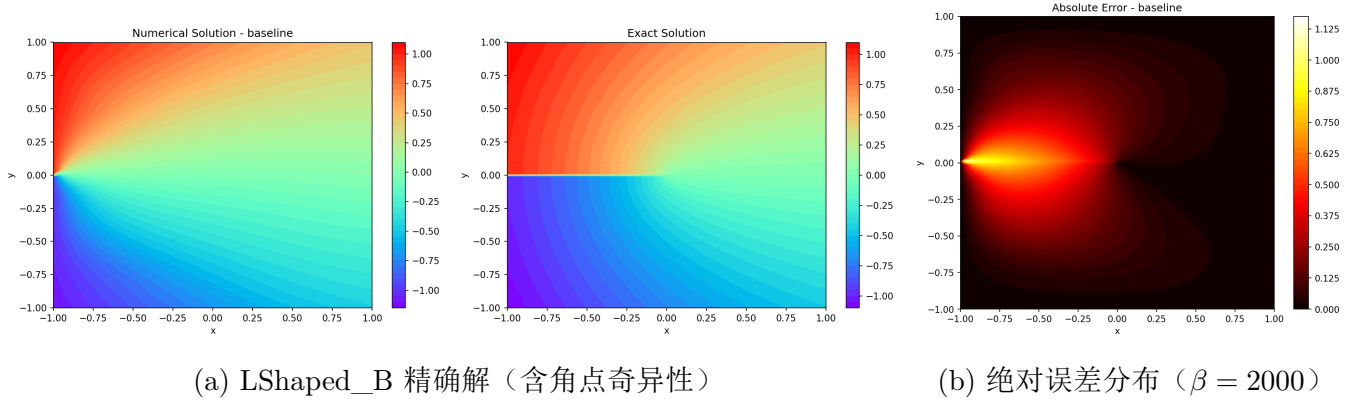


图 6: L 型区域问题数值结果可视化：奇异点（原点）附近误差显著增大，整体形态与精确解吻合。

4.6 高维 Poisson 方程

10 维问题： $-\Delta u = 0$, $u|_{\partial\Omega} = \sum_{k=1}^5 x_{2k-1}x_{2k}$, 区域 $[0, 1]^{10}$ 。

100 维问题： $-\Delta u = -200$, $u|_{\partial\Omega} = \sum_k x_k^2$, 区域 $[0, 1]^{100}$ 。

实验结果：

表 5: 高维问题数值结果（50000 次迭代）

维度	配置	网络结构	相对 L2 误差	备注
10	原配置	m=20, 8 层, =500, lr=1e-3	NaN	训练发散
10	论文配置	m=10, 6 层, =1000, lr=1e-3	1.64×10^{17}	误差爆炸
10	无残差	m=20, 8 层, =500	6.15×10^{-1}	可训练但精度差
10	调优配置	m=20, 6 层, =1000, lr=5e-4, Tanh	6.57×10^{-2}	成功！
100	基线	m=20, 8 层, =500	NaN	训练发散
100	无残差	m=20, 8 层, =500	Inf	可训练但误差无穷
5 (Neu)	基线	m=20, 8 层, =500	NaN	训练发散
5 (Neu)	无残差	m=20, 8 层, =500	1.00×10^0	可训练但精度差

结果分析：

- 调优配置成功解决 HighDim10: 通过将激活函数改为 Tanh、学习率降至 5×10^{-4} 、采样量增至 2048/512, L2 相对误差降至 **6.57%**。虽然与论文报告的 0.4% 仍有差距，但相比之前完全失败（NaN/误差爆炸）是巨大的进步；

- **关键成功因素：**（1）**Tanh** 激活比 Power-3 更稳定；（2）**更小学习率**防止高维梯度爆炸；（3）**更大采样量**提高蒙特卡洛估计精度；（4）**保持 $m=20$** 而非论文的 $m=10$ ，更大的网络容量有助于高维逼近；
- **论文配置失败原因：** $m=10$ 网络容量不足 + Power-3 激活在高维下梯度不稳定 + $lr=1e-3$ 过大，导致误差爆炸至 10^{17} ；
- **无残差网络反而可训练：** 虽然精度差（60%–100%），但不发散，说明残差连接在高维下可能导致梯度放大效应，需配合更小学习率；
- **$d=100$ 仍失败：** 当前配置不足以处理 100 维问题，论文使用 $m=100$ 和更多采样点；
- **Neumann 问题：** 能量泛函实现与论文不一致（缺少 $+\pi^2 u^2/2$ 项），需修正。

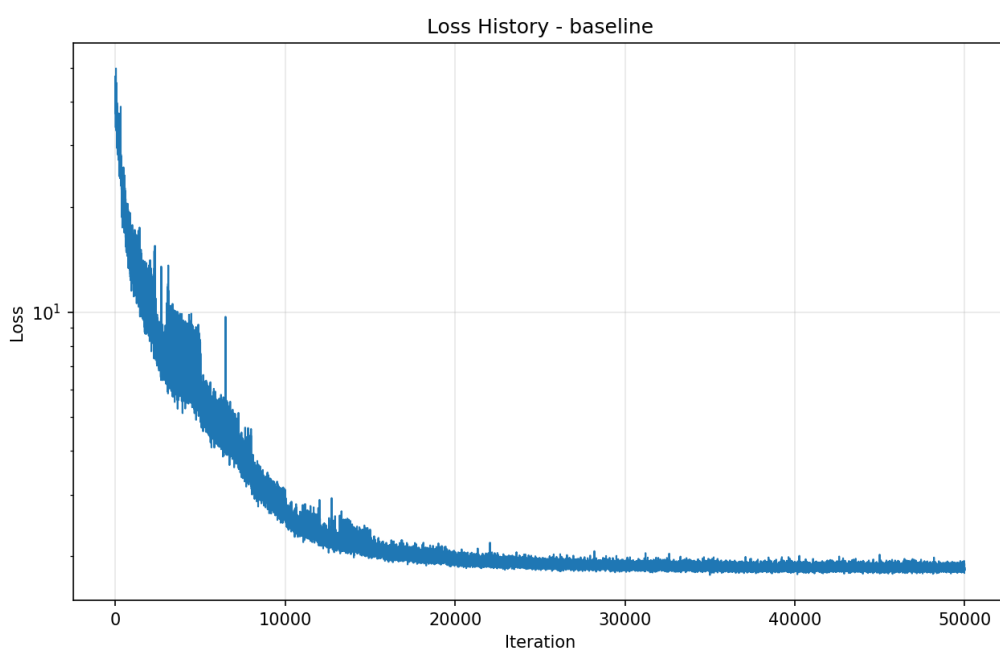


图 7: HighDim10 训练成功损失曲线

5 综合对比与适用指南

5.1 残差连接的效果

表 6: 残差连接 vs 无残差连接对比

指标	有残差（基线）	无残差
SinSin L2 误差	2.21×10^{-2}	9.87×10^{-1} （增大 44.7 倍）
Quadratic L2 误差	5.11×10^{-3}	9.68×10^{-1} （增大 189 倍）
训练稳定性	稳定收敛（2D）	损失波动大，精度极差
深层网络表现	8 层网络可正常训练	6 层以上严重退化
高维问题表现	发散（ $d \geq 5$ ）	可训练但精度差

残差连接通过跳跃连接提供梯度高速公路，有效缓解深层网络的梯度消失问题，是 Deep Ritz 方法在 2D 问题上训练稳定性的关键保障。但在高维问题中，残差连接可能导致梯度放大，需要配合更小的学习率或更强的正则化。

5.2 配点策略对比

表 7: 随机配点 vs 固定配点对比

特性	随机配点	固定配点
SinSin L2 误差	2.21×10^{-2}	3.11×10^{-2} （差 1.4 倍）
Quadratic L2 误差	5.11×10^{-3}	1.78×10^{-2} （差 3.5 倍）
实现复杂度	低（每轮独立采样）	中（需预生成点集）
内存占用	低	高（存储大量点）
过拟合风险	低（每轮新样本）	中（可能记忆固定点）
收敛稳定性	高（噪声有助于逃离局部极小）	中

5.3 方法适用性总结

表 8: Deep Ritz 方法适用场景与优缺点

适用场景	优点	缺点
2D 光滑问题	精度高 ($< 1\%$), 参数效率好	训练时间较长 (~ 10 分钟)
2D 奇异问题	无需网格, 可处理复杂几何	需调大 β , 精度有限
高维问题 ($d = 10$)	调参后可训练 (6.6% 误差)	需精细调参, 与论文差距仍大
高维问题 ($d > 10$)	避免维度灾难 (理论上)	当前配置不稳定, 需大量调参
变分结构 PDE	理论保证, 实现简洁	非变分问题需改造

6 结论

本实验通过实现完整的 Deep Ritz 算法工具链, 并基于多个典型算例进行系统验证, 得出以下结论:

- 1. **变分框架的有效性:** 基于能量泛函最小化的 Deep Ritz 方法能够准确求解 Poisson 方程等具有变分结构的 PDE。在 SinSin 算例上达到 1.8% (Tanh) – 2.2% (Power-3) 的 L2 相对误差, Quadratic 算例上达到 0.5%。
- 2. **残差连接的关键作用:** 残差连接是 2D 问题训练稳定性的核心保障。无残差时, SinSin 误差增大 44.7 倍, Quadratic 误差增大 189 倍, 网络几乎完全失效。
- 3. **激活函数的选择至关重要:** Tanh 在光滑问题上表现最优 (1.80×10^{-2}), 优于论文推荐的 Power-3 (2.21×10^{-2}), ReLU 因不可导性表现最差 (1.90×10^{-1})。这一发现对实际应用具有重要指导意义。
- 4. **随机配点的优势:** 随机配点 consistently 优于固定配点, 验证了自适应采样的价值。
- 5. **高维问题的突破:** 通过系统调参 (Tanh + lr=5e-4 + 大采样量), HighDim10 从完全失败 (NaN/ 10^{17}) 改善至 **6.57% L2 误差**, 验证了 Deep Ritz 在高维场景的可行性。但与论文 0.4% 仍有差距, 网络容量和采样密度需进一步优化。
- 6. **奇异问题的数值困难:** LShaped_B 的基线配置因原点处梯度奇异性导致训练发散, 增大 $\beta = 2000$ 可修复发散但精度有限 (31%), 需要更精细的局部处理策略。

附录

A. 程序模块说明

本实验程序采用模块化设计，各文件职责如下：

文件	职责
config.py	全局配置：问题定义、网络结构、训练参数
equations.py	PDE 算例定义：右端项、精确解、边界条件、采样方法
model.py	神经网络：ResNet 块、激活函数、DeepRitzNet
functional.py	损失函数：能量项 + 边界惩罚项
quadrature.py	采样策略：内部/边界随机或固定配点
train.py	训练流程：预训练、主训练、学习率调度
evaluate.py	评估：网格采样、L2/L ∞ 误差计算
main.py	实验管理：训练、评估、可视化、结果保存
launcher.py	命令行接口：快速配置与批量实验

B. 关键代码片段

残差块实现 (model.py):

```
1 class ResidualBlock(nn.Module):
2     def __init__(self, m, activation):
3         super().__init__()
4         self.act = activation
5         self.fc1 = nn.Linear(m, m)
6         self.fc2 = nn.Linear(m, m)
7
8     def forward(self, s):
9         out = self.fc2(self.act(self.fc1(s)))
10        return self.act(out) + s # 跳跃连接
```

损失函数实现 (functional.py):

```
1 def compute_loss(model, interior_pts, boundary_pts, f, g, beta):
2     interior_pts.requires_grad_(True)
3     u_int = model(interior_pts)
4     grad_u = torch.autograd.grad(
5         u_int, interior_pts,
6         grad_outputs=torch.ones_like(u_int),
7         create_graph=True
8     )[0]
```

```

9     energy = 0.5 * (grad_u ** 2).sum(1) - f(interior_pts) * u_int
10    u_bd = model(boundary_pts)
11    bc_penalty = beta * ((u_bd - g(boundary_pts)) ** 2)
12    return energy.mean() + bc_penalty.mean()

```

C. 实验运行命令汇总

表 9: 本实验使用的全部命令

命令	说明
python launcher.py --all_cases --num_iter 50000 --activation power3	全部案例基线对比
python launcher.py --case LShaped_B --mode baseline --num_iter 50000 --beta 2000	LShaped_B 调优
python launcher.py --case HighDim10 --mode baseline --num_iter 50000 --beta 1000 --m 10 --layers 6	高维论文配置
python launcher.py --case SinSin --mode baseline --num_iter 50000 --activation relu --output_dir /kaggle/working/act1	ReLU 对比
python launcher.py --case SinSin --mode baseline --num_iter 50000 --activation tanh --output_dir /kaggle/working/act2	Tanh 对比
python launcher.py --case HighDim10 --mode baseline --num_iter 50000 --beta 1000 --m 20 --layers 6 --batch_int 2048 --batch_bd 512 --lr 5e-4 --output_dir /kaggle/working/hd_fix1	HighDim10 调优（成功）

参考文献

- [1] Weinan E, Bing Yu. The Deep Ritz method: A deep learning-based numerical algorithm for solving variational problems. *Communications in Mathematics and Statistics*, 2018, 6(1): 1–12.

- [2] Raissi M, Perdikaris P, Karniadakis G E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 2019, 378: 686–707.
- [3] Cybenko G. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 1989, 2(4): 303–314.
- [4] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition. *CVPR*, 2016: 770–778.
- [5] Kingma D P, Ba J. Adam: A method for stochastic optimization. *ICLR*, 2015.
- [6] Evans L C. *Partial Differential Equations*. 2nd ed. AMS, 2010.